

LAPORAN TUGAS BESAR

IF25-21013 STRATEGI ALGORITMA | Semester Genap 2026/2027

Pemanfaatan Algoritma *Greedy* dalam Pembuatan *Bot* Permainan Robocode Tank Royale



Foto Anggota Kelompok

Nama Kelompok : *Kacauuw*
Anggota : 1. Jelita Ayu Ressito Siregar (124140056)
2. Dintani Metha Hamdi (124140116)
3. Naura Aziza Nurhanifa (124140072)
Kelas : RA
Dosen : Imam Eko Wicaksono, S.Si., M.Si.

Program Studi Teknik Informatika
Fakultas Teknologi Industri
Institut Teknologi Sumatera

2026

Daftar Isi

1	Deskripsi Tugas	1
1.1	Spesifikasi Tugas	1
2	Landasan Teori	3
2.1	Dasar Teori Algoritma Greedy Secara Umum	3
2.1.1	Karakteristik Algoritma <i>Greedy</i>	3
2.1.2	Elemen-Elemen Algoritma <i>Greedy</i>	3
2.2	Cara Kerja Program Secara Umum	4
2.2.1	Mekanisme Bot Melakukan Aksi	4
2.2.2	Implementasi Algoritma Greedy ke Dalam Bot	5
3	Aplikasi Strategi <i>Greedy</i>	6
3.1	Proses Mapping Persoalan ke Elemen Algoritma <i>Greedy</i>	6
3.2	Eksplorasi 4 Alternatif Solusi Greedy	6
3.2.1	Bot 1: [Aggressive Hunter]	6
3.2.2	Bot 2: [Energy Saver]	7
3.2.3	Bot 3: [Dodge and Counter]	7
3.2.4	Bot 4: [Predictive Shooter (Bot Utama)]	7
3.3	Analisis Efisiensi dan Efektivitas Alternatif Solusi	8
3.4	Strategi Greedy yang Dipilih Beserta Alasan Pemilihan	8
4	Implementasi dan Pengujian	9
4.1	Implementasi 4 Alternatif Solusi (Pseudocode)	9
4.1.1	Pseudocode Bot 1: Evasive Movement Greedy	9
4.1.2	Pseudocode Bot 2: Aggressive Chaos Greedy	11
4.1.3	Pseudocode Bot 3: Survival Sniper Greedy	13
4.1.4	Pseudocode Bot 4: Adaptive Pressure Greedy – Bot Utama	14
4.2	Struktur Program Bot Utama yang Dipilih	17
4.2.1	Struktur Data	17
4.2.2	Fungsi dan Prosedur yang Digunakan	18
4.3	Pengujian Laga 1 vs 1 dengan Bot Asisten	18

4.3.1	Konfigurasi Lingkungan Pengujian	19
4.3.2	Data Hasil Rekapitulasi Match Pengujian	19
4.3.3	Catatan Kejadian Unik Selama Pertandingan	19
4.4	Analisis Hasil Pengujian	20
5	Kesimpulan dan Saran	22
5.1	Kesimpulan	22
5.2	Saran	22
A	Lampiran	24
A.1	Tautan Repository GitHub	24

Daftar Gambar

Daftar Tabel

3.1	Mapping Robocode Tank Royale ke Elemen Algoritma <i>Greedy</i> . . .	6
3.2	Perbandingan Efisiensi dan Efektivitas Alternatif Solusi	8
4.1	Daftar Komponen Fungsi dan Prosedur Kerja Bot Utama	18
4.2	Rekap Hasil Komponen Skor Pengujian 1 s.d 3	19

BAB 1 Deskripsi Tugas

1.1 Spesifikasi Tugas

- Buatlah 4 *bot* (1 utama dan 3 alternatif, tetap dikumpulkan 1 *bot* yang terbaik saja) dalam bahasa C# (.NET) yang mengimplementasikan algoritma *Greedy* pada *bot* permainan Robocode Tank Royale dengan tujuan memenangkan permainan.
- Tugas dikerjakan berkelompok dengan anggota minimal 2 orang dan maksimal 3 orang.
- Strategi *greedy* yang diimplementasikan setiap kelompok harus dikaitkan dengan fungsi objektif dari permainan ini, yaitu memperoleh skor setinggi mungkin pada akhir pertempuran. Hal ini dapat dilakukan dengan mengoptimalkan komponen skor yang telah dijelaskan di atas.
- Strategi *greedy* yang diimplementasikan harus berbeda untuk setiap *bot* yang diimplementasikan dan setiap strategi *greedy* harus menggunakan *heuristic* yang berbeda.
- *Bot* yang dibuat **TIDAK BOLEH** sama dengan **SAMPEL** yang diberikan sebagai **CONTOH**, baik dari *starter pack* maupun dari *repository engine* asli.
- Buatlah strategi *greedy* terbaik, karena setiap *bot* utama dari masing-masing kelompok akan diadu dalam kompetisi Tugas Besar.
- Strategi *greedy* yang kelompok Anda buat harus dijelaskan dan ditulis secara eksplisit pada laporan, karena akan diperiksa saat demo apakah strategi yang dituliskan sesuai dengan yang diimplementasikan.
- Setiap kelompok dapat menggunakan kreativitas yang bermacam-macam dalam menyusun strategi *greedy* untuk memenangkan permainan. Implementasi pemain harus dapat dijalankan pada *game engine* yang telah disebutkan di atas serta dapat dikompetisikan dengan *bot* dari kelompok lain.
- Program harus mengandung komentar yang jelas, dan untuk setiap strategi *greedy* yang disebutkan, harus dilengkapi dengan kode sumber yang dibuat. Artinya, semua strategi harus diimplementasikan.

- Mahasiswa disarankan membaca dokumentasi dari *game engine*. Perlu diperhatikan bahwa *game engine* yang digunakan untuk Tugas Besar ini **SUDAH DIMODIFIKASI**. Untuk perubahan dapat dilihat pada bagian *starter pack*.

BAB 2 Landasan Teori

2.1 Dasar Teori Algoritma Greedy Secara Umum

Algoritma *Greedy* (serakah) merupakan kelas fundamental dalam algoritma matematika dan ilmu komputer, yang didefinisikan melalui pendekatan iteratifnya dalam mengambil keputusan optimal secara lokal untuk mendekati solusi global optimal. Dalam konteks pembuatan bot untuk Robocode Tank Royale, algoritma greedy akan memandu bot untuk mengambil keputusan terbaik pada setiap turn (misalnya, menentukan arah gerak, target tembak, atau daya tembak) berdasarkan situasi saat itu, tanpa mempertimbangkan konsekuensi jangka panjang, dengan tujuan akhir memaksimalkan akumulasi skor sepanjang pertempuran.

2.1.1 Karakteristik Algoritma *Greedy*

Terdapat beberapa karakteristik utama yang membedakan algoritma greedy dari kelas algoritma lainnya. Pertama, algoritma greedy bersifat iteratif, artinya solusi dibangun secara bertahap langkah demi langkah. Kedua, algoritma greedy menerapkan prinsip pilihan optimal lokal (*locally optimal choice*), yaitu pada setiap langkah, pilihan yang diambil adalah pilihan terbaik berdasarkan informasi yang tersedia saat itu. Ketiga, keputusan yang telah diambil dalam algoritma greedy bersifat irreversible atau tidak dapat diubah kembali, sehingga setiap langkah memiliki konsekuensi yang permanen terhadap solusi akhir yang dihasilkan.

2.1.2 Elemen-Elemen Algoritma *Greedy*

Algoritma *Greedy* secara umum terdiri dari elemen-elemen berikut:

- **Himpunan Kandidat:** Himpunan kandidat adalah semua aksi yang mungkin dilakukan bot pada setiap turn, seperti: bergerak maju, bergerak mundur, memutar body ke kiri/kanan, memutar turret, memutar radar, menembak dengan daya tembak tertentu (0.1 - 3.0), atau kombinasi dari aksi-aksi tersebut.
- **Fungsi Seleksi:** Fungsi seleksi adalah fungsi yang pada setiap langkah memilih kandidat yang paling memungkinkan untuk mencapai solusi optimal. Fungsi ini merupakan inti dari algoritma greedy karena menentukan pilihan mana yang dianggap "paling menguntungkan" pada saat itu.

- **Fungsi Kelayakan:** memeriksa apakah kandidat yang dipilih dapat dimasukkan ke dalam solusi tanpa melanggar batasan.
- **Fungsi Objektif:** Fungsi objektif adalah fungsi yang memaksimalkan atau meminimumkan nilai solusi. Dalam konteks algoritma greedy, fungsi objektif menjadi tolok ukur seberapa baik suatu solusi, dan algoritma akan berusaha mendekati nilai optimal dari fungsi ini.
- **Fungsi Solusi:** menentukan kapan solusi dianggap lengkap.
- **Himpunan Solusi:** adalah kumpulan kandidat yang telah dipilih dan membentuk solusi parsial.

2.2 Cara Kerja Program Secara Umum

Secara umum, bot pada permainan strategi real-time bekerja dalam siklus berulang yang dieksekusi setiap giliran (turn). Pada setiap siklus, bot menerima informasi terkini tentang kondisi lingkungan seperti posisi diri, posisi musuh, dan status energi, kemudian memproses informasi tersebut melalui logika yang telah ditentukan untuk menghasilkan serangkaian perintah aksi.

Dalam konteks permainan dengan batasan waktu per giliran, bot tidak memiliki kemewahan untuk menelusuri seluruh ruang kemungkinan aksi. Oleh karena itu, bot harus mampu membuat keputusan secara cepat berdasarkan kondisi saat itu saja. Pendekatan greedy sangat sesuai untuk skenario ini karena cukup mengevaluasi sekumpulan kandidat aksi yang tersedia, memilih yang terbaik berdasarkan fungsi seleksi, lalu langsung mengeksekusinya tanpa perlu menunggu hasil kalkulasi jangka panjang.

Implementasi algoritma greedy ke dalam bot dilakukan dengan memetakan setiap kondisi permainan ke dalam fungsi seleksi yang menentukan aksi mana yang memberikan nilai heuristik tertinggi pada giliran tersebut. Aksi yang dipilih kemudian dikirimkan ke game engine sebagai perintah untuk giliran berjalan, dan siklus ini berulang hingga pertempuran selesai.

2.2.1 Mekanisme Bot Melakukan Aksi

Bot pada permainan Robocode Tank Royale bekerja berdasarkan sistem turn yang berjalan secara terus menerus selama pertandingan berlangsung. Pada setiap turn, bot akan melakukan pemindaian menggunakan radar untuk mendeteksi keberadaan

musuh di sekitar arena. Informasi hasil pemindaian seperti posisi, arah gerak, jarak, dan energi musuh kemudian digunakan sebagai dasar pengambilan keputusan.

Ketika musuh berhasil terdeteksi, bot akan menentukan aksi terbaik yang dapat dilakukan pada saat itu. Aksi tersebut dapat berupa bergerak menghindari peluru, mendekati lawan, menjaga jarak aman, memutar turret ke arah target, ataupun melakukan penembakan dengan kekuatan tertentu. Seluruh keputusan diambil secara real-time berdasarkan kondisi arena saat pertandingan berlangsung.

Selain aksi menyerang, bot juga memiliki mekanisme bertahan seperti menghindari tabrakan dengan dinding arena, mengubah arah gerak secara berkala, serta melakukan repositioning ketika energi bot berada dalam kondisi rendah. Dengan mekanisme tersebut, bot dapat mempertahankan keberlangsungan hidup lebih lama selama pertandingan.

2.2.2 Implementasi Algoritma Greedy ke Dalam Bot

Implementasi algoritma greedy pada bot dilakukan dengan cara memilih aksi yang dianggap paling menguntungkan pada setiap turn pertandingan. Bot tidak melakukan perhitungan jangka panjang terhadap seluruh kemungkinan kondisi permainan, melainkan hanya memilih keputusan lokal terbaik berdasarkan informasi yang tersedia saat itu.

Strategi greedy diterapkan pada beberapa aspek utama permainan seperti pemilihan target, penentuan arah gerak, prioritas penghindaran peluru, serta pengaturan kekuatan tembakan. Setiap keputusan dibuat menggunakan heuristic tertentu yang berbeda pada masing-masing alternatif bot.

Sebagai contoh, bot dapat memilih menyerang musuh terdekat untuk meningkatkan peluang tembakan mengenai target, atau memilih bergerak menjauh ketika energi bot lebih rendah dibandingkan lawan. Pendekatan greedy memungkinkan bot mengambil keputusan dengan cepat sehingga cocok diterapkan pada permainan yang berjalan secara dinamis dan real-time.

BAB 3 Aplikasi Strategi *Greedy*

3.1 Proses Mapping Persoalan ke Elemen Algoritma *Greedy*

Berikut adalah hasil pemetaan komponen mekanis dan sistem skor *Robocode Tank Royale* ke dalam enam elemen dasar pembentuk algoritma *Greedy*:

Tabel 3.1: Mapping Robocode Tank Royale ke Elemen Algoritma *Greedy*

Elemen Greedy	Pemetaan ke Robocode Tank Royale
Himpunan Kandidat	Pilihan aksi yang mungkin dieksekusi pada setiap <i>turn</i> : arah sudut gerak, sudut putar <i>turret/radar</i> , intensitas <i>fire power</i> , dsb.
Himpunan Solusi	Rangkaian keputusan aksi yang berhasil dieksekusi dari <i>turn</i> awal hingga <i>turn</i> terakhir.
Fungsi Seleksi	Penentuan satu aksi spesifik menggunakan rumusan *heuristic* tertentu yang dinilai memberikan keuntungan instan terbesar saat <i>turn</i> berjalan.
Fungsi Kelayakan	Validasi batas fisik *environment*: tidak menabrak dinding batas arena, nilai temperatur <i>gun</i> siap tembak (≤ 0), sisa energi mencukupi, dll.
Fungsi Objektif	Memaksimalkan akumulasi metrik skor akhir pertempuran (<i>Bullet Damage</i> , <i>Survival Score</i> , <i>Ram Damage Bonus</i> , dll).
Fungsi Solusi	Kondisi akhir pertempuran terpenuhi: hanya tersisa satu <i>bot</i> pemenang di arena atau batasan <i>Turn Limit</i> ronde telah habis.

3.2 Eksplorasi 4 Alternatif Solusi Greedy

Setiap alternatif rancangan di bawah dirancang menggunakan dasar *heuristic* yang unik dan berbeda guna menguji responsivitas performa komponen nilai objektif yang ingin dikejar.

3.2.1 Bot 1: [Aggressive Hunter]

Deskripsi Heuristik: Bot menggunakan heuristic penyerangan agresif dengan memprioritaskan musuh terdekat sebagai target utama. Semakin dekat jarak musuh, semakin tinggi prioritas penyerangan yang diberikan.

Fungsi Objektif yang Dioptimalkan: Memaksimalkan *Bullet Damage* dan *Bullet Damage Bonus*.

Cara Kerja: Bot akan terus melakukan *scanning* terhadap arena menggunakan radar. Ketika musuh ditemukan, bot langsung memutar turret menuju target dan menembakkan peluru dengan kekuatan sedang hingga besar. Selain menyerang, bot juga bergerak mendekati lawan untuk menjaga peluang tembakan tetap tinggi.

3.2.2 Bot 2: [Energy Saver]

Deskripsi Heuristik: Bot menerapkan heuristic penghematan energi dengan mempertimbangkan kondisi energi bot sebelum melakukan penembakan.

Fungsi Objektif yang Dioptimalkan: Memaksimalkan *Survival Score* dengan menjaga energi tetap stabil.

Cara Kerja: Bot hanya menggunakan tembakan dengan *power* besar ketika peluang mengenai target cukup tinggi. Jika energi bot rendah, bot akan lebih banyak bergerak menghindari dan mengurangi intensitas penembakan. Strategi ini bertujuan memperpanjang waktu bertahan hidup di arena.

3.2.3 Bot 3: [Dodge and Counter]

Deskripsi Heuristik: Bot memprioritaskan pergerakan menghindari untuk meminimalkan *damage* yang diterima sebelum melakukan serangan balasan.

Fungsi Objektif yang Dioptimalkan: Memaksimalkan *Survival Score* dan mengurangi *damage* yang diterima.

Cara Kerja: Bot bergerak secara acak atau zig-zag untuk mempersulit lawan dalam membidik target. Ketika musuh menembak atau berada pada posisi tertentu, bot akan segera mengubah arah gerak. Setelah posisi dianggap aman, bot melakukan serangan balik menggunakan turret.

3.2.4 Bot 4: [Predictive Shooter (Bot Utama)]

Deskripsi Heuristik: Bot menggunakan heuristic prediksi pergerakan lawan dengan memperkirakan arah gerak musuh sebelum menembakkan peluru.

Fungsi Objektif yang Dioptimalkan: Memaksimalkan akurasi tembakan dan total skor pertandingan.

Cara Kerja: Bot melakukan *scan* terhadap posisi dan arah pergerakan musuh secara kontinu. Berdasarkan data tersebut, bot memperkirakan posisi target beberapa saat ke depan lalu mengarahkan turret ke titik prediksi tersebut. Selain menyerang, bot tetap bergerak untuk menghindari serangan lawan.

3.3 Analisis Efisiensi dan Efektivitas Alternatif Solusi

Evaluasi teoretis tingkat kompleksitas komputasi tiap kali siklus *turn* berjalan beserta potensi efektivitas performanya di arena:

Tabel 3.2: Perbandingan Efisiensi dan Efektivitas Alternatif Solusi

Bot	Kompleksitas / Turn	Kelebihan	Kekurangan
Bot 1	$O(1)$	Agresif dan mampu menghasilkan <i>damage</i> besar	Boros energi dan rentan terkena serangan balik
Bot 2	$O(1)$	Daya tahan lebih baik karena hemat energi	<i>Damage</i> yang dihasilkan relatif lebih kecil
Bot 3	$O(1)$	Sulit ditembak oleh lawan	Akurasi tembakan menurun saat bergerak terus menerus
Bot 4	$O(1)$	Akurasi tembakan lebih tinggi dan performa lebih stabil	Membutuhkan perhitungan posisi musuh secara kontinu

3.4 Strategi Greedy yang Dipilih Beserta Alasan Pemilihan

Berdasarkan hasil komparasi, strategi yang diputuskan untuk diimplementasikan secara penuh pada bot utama kompetisi adalah **Bot [X]: [Nama Strategi]**.

Alasan dan Pertimbangan Pemilihan:

- Memiliki akurasi tembakan yang lebih baik dibandingkan strategi lainnya.
- Mampu menghasilkan total skor yang lebih stabil pada beberapa pertandingan.
- Dapat menyeimbangkan kemampuan menyerang dan bertahan.
- Memiliki performa yang cukup efektif terhadap berbagai pola gerakan lawan.

BAB 4 Implementasi dan Pengujian

4.1 Implementasi 4 Alternatif Solusi (Pseudocode)

4.1.1 Pseudocode Bot 1: Evasive Movement Greedy

```
1
2 CLASS VelocityBot
3
4 VARIABLE moveDirection = 1
5
6 FUNCTION Main(args)
7 BEGIN
8     new VelocityBot().Start()
9 END
10
11
12 FUNCTION Run()
13 BEGIN
14     BodyColor = Green
15     GunColor = Lime
16     RadarColor = Magenta
17     BulletColor = Aqua
18     TracksColor = Yellow
19     ScanColor = Cyan
20
21     WHILE IsRunning DO
22
23         TurnRadarRight(360)
24
25         Forward(150 * moveDirection)
26
27         TurnRight(40)
28
29         moveDirection = moveDirection * -1
30
31     END WHILE
```

```
32 END
33
34
35 FUNCTION OnScannedBot(e)
36 BEGIN
37
38     IF Energy > 60 THEN
39
40         Fire(3)
41
42     ELSE IF Energy > 30 THEN
43
44         Fire(2)
45
46     ELSE
47
48         Fire(1)
49
50     END IF
51
52     TurnRight(20)
53     Forward(100 * moveDirection)
54
55 END
56
57
58 FUNCTION OnHitWall(e)
59 BEGIN
60
61     moveDirection = moveDirection * -1
62
63     Back(120)
64     TurnRight(90)
65
66 END
67
68
69 FUNCTION OnHitBot(e)
70 BEGIN
71
```

```
72     Fire(3)
73
74     TurnRight(45)
75     Back(80)
76
77 END
78
79
80 FUNCTION OnHitByBullet(e)
81 BEGIN
82
83     moveDirection = moveDirection * -1
84
85     TurnLeft(60)
86     Forward(120)
87
88 END
89
90 END CLASS
```

Listing 4.1: Detail Logika Implementasi Alternatif Bot 1

4.1.2 Pseudocode Bot 2: Aggressive Chaos Greedy

```
1 PROGRAM KacauuwBot
2
3 VARIABLE:
4     moveDirection = 1
5
6 FUNCTION Run():
7     SET AdjustGunForBodyTurn = TRUE
8     SET AdjustRadarForGunTurn = TRUE
9
10    WHILE IsRunning DO
11
12        TurnRadarRight(360)
13
14
15        Forward(200 * moveDirection)
16        TurnRight(35)
```



```
17
18
19     IF Energy < 40 THEN
20         moveDirection = moveDirection * -1
21     END IF
22 END WHILE
23
24 FUNCTION OnScannedBot(e):
25     Forward(120)
26
27     IF Energy > 60 THEN
28         Fire(3)
29     ELSE IF Energy > 25 THEN
30         Fire(2)
31     ELSE
32         Fire(1)
33     END IF
34
35     TurnRight(20)
36
37 FUNCTION OnHitByBullet(e):
38     moveDirection = moveDirection * -1
39     TurnRight(90)
40     Forward(150)
41
42 FUNCTION OnHitWall(e):
43     Back(120)
44     TurnRight(90)
45     moveDirection = moveDirection * -1
46
47 FUNCTION OnHitBot(e):
48     Fire(3)
49     Forward(80)
50     TurnRight(30)
51
52 END PROGRAM
```

Listing 4.2: Detail Logika Implementasi Alternatif Bot 2

4.1.3 Pseudocode Bot 3: Survival Sniper Greedy

```
1
2 CLASS PlengerBot
3
4 FUNCTION Main(args)
5 BEGIN
6     new PlengerBot().Start()
7 END
8
9
10 FUNCTION Run()
11 BEGIN
12     BodyColor = Blue
13     GunColor = DarkBlue
14     RadarColor = Cyan
15     BulletColor = LightBlue
16     TracksColor = Silver
17     ScanColor = Aqua
18
19     WHILE IsRunning DO
20
21         TurnRadarRight(360)
22
23         Back(50)
24         TurnRight(30)
25         Forward(80)
26         TurnLeft(30)
27
28     END WHILE
29 END
30
31
32 FUNCTION OnScannedBot(e)
33 BEGIN
34
35     Fire(1)
36
37 END
38
```

```
39
40 FUNCTION OnHitWall(e)
41 BEGIN
42
43     Back(100)
44     TurnRight(90)
45
46 END
47
48
49 FUNCTION OnHitBot(e)
50 BEGIN
51
52     Back(50)
53     Fire(2)
54
55 END
56
57 END CLASS
```

Listing 4.3: Detail Logika Implementasi Alternatif Bot 3

4.1.4 Pseudocode Bot 4: Adaptive Pressure Greedy – Bot Utama

```
1
2 CLASS TolakAnginBot
3
4 FUNCTION Main(args)
5 BEGIN
6     new TolakAnginBot().Start()
7 END
8
9
10 FUNCTION Run()
11 BEGIN
12     BodyColor = Blue
13     GunColor = Navy
14     RadarColor = Yellow
15     BulletColor = LightPink
16     TracksColor = Pink
```

```
17     ScanColor = Purple
18
19     WHILE IsRunning DO
20
21         TurnRadarRight(360)
22
23         IF Energy > 60 THEN
24
25             Forward(150)
26             TurnRight(20)
27
28         ELSE IF Energy > 30 THEN
29
30             Forward(80)
31             TurnLeft(40)
32
33         ELSE
34
35             Back(120)
36             TurnRight(90)
37
38         END IF
39
40     END WHILE
41 END
42
43
44 FUNCTION OnScannedBot(e)
45 BEGIN
46
47     IF Energy > 70 THEN
48
49         Fire(3)
50
51     ELSE IF Energy > 40 THEN
52
53         Fire(2)
54
55     ELSE
56
```

```
57         Fire(1)
58
59     END IF
60
61 END
62
63
64 FUNCTION OnHitWall(e)
65 BEGIN
66
67     Back(100)
68     TurnRight(90)
69
70 END
71
72
73 FUNCTION OnHitBot(e)
74 BEGIN
75
76     Fire(3)
77     Back(60)
78
79 END
80
81
82 FUNCTION OnHitByBullet(e)
83 BEGIN
84
85     TurnRight(45)
86
87     IF Energy > 50 THEN
88
89         Fire(3)
90
91     ELSE IF Energy > 25 THEN
92
93         Fire(2)
94
95     ELSE
96
```

```
97         Fire(1)
98
99     END IF
100
101     Forward(100)
102
103 END
104
105 END CLASS
```

Listing 4.4: Detail Logika Implementasi Alternatif Bot 4 (Utama)

4.2 Struktur Program Bot Utama yang Dipilih

Bagian ini menguraikan detail modul teknis dari kode program solusi *greedy* terpilih.

4.2.1 Struktur Data

Pada implementasi bot Robocode Tank Royale, struktur data yang digunakan relatif sederhana karena strategi *greedy* yang diterapkan lebih berfokus pada pengambilan keputusan secara langsung berdasarkan kondisi saat ini. Data utama yang digunakan berasal dari state bawaan engine Robocode Tank Royale seperti posisi bot, energi, arah gerak, serta informasi musuh yang diperoleh melalui event scanning.

Beberapa data penting yang digunakan antara lain:

- **Energy**

Digunakan untuk menentukan tingkat agresivitas bot. Nilai energi digunakan sebagai acuan dalam menentukan movement dan besar firepower yang digunakan.

- **moveDirection**

Variabel integer yang digunakan untuk menentukan arah gerak bot. Nilai ini biasanya bernilai 1 atau -1 untuk mengubah arah movement sehingga bot tidak mudah diprediksi lawan.

- **Koordinat Arena**

Posisi bot direpresentasikan menggunakan koordinat X dan Y yang disediakan oleh engine Robocode. Data ini digunakan untuk mendeteksi posisi bot terhadap arena dan membantu movement agar tidak mudah terjebak di sudut arena.

- **Event ScannedBotEvent**

Struktur data event ini digunakan untuk menyimpan informasi musuh yang terdeteksi radar. Informasi tersebut digunakan untuk menentukan kapan bot menembak serta seberapa besar firepower yang digunakan.

- **Event HitByBulletEvent**

Digunakan untuk mendeteksi ketika bot terkena peluru lawan. Event ini memicu movement evasive atau dodge movement untuk mengurangi kemungkinan terkena peluru berikutnya.

- **Event HitWallEvent dan HitBotEvent**

Digunakan untuk menangani kondisi ketika bot menabrak dinding arena atau bertabrakan dengan bot lain. Data event ini membantu bot melakukan repositioning agar movement tetap berjalan efektif.

Secara umum, struktur data yang digunakan bersifat lightweight dan sederhana agar proses pengambilan keputusan tetap cepat sehingga bot dapat berjalan stabil pada batas waktu turn timeout yang diberikan.

4.2.2 Fungsi dan Prosedur yang Digunakan

Tabel 4.1: Daftar Komponen Fungsi dan Prosedur Kerja Bot Utama

Nama Fungsi/Prosedur	Deskripsi Penggunaan
OnScannedBot()	Event handler otomatis saat radar mendeteksi posisi musuh di area cakupan scan.
OnHitByBullet()	Penanganan keadaan darurat ketika bot menerima damage peluru musuh.
OnHitWall()	Prosedur penyesuaian arah rotasi roda untuk menghindari malfungsi akibat benturan dinding arena.
OnHitBot()	Handler kalkulasi ramming ketika terjadi kontak tabrakan fisik langsung antar bot.
Run()	Main loop siklus eksekusi utama bot yang berjalan kontinu di setiap *turn*.

4.3 Pengujian Laga 1 vs 1 dengan Bot Asisten

Pengujian disimulasikan minimal sebanyak 3 kali pertandingan dengan skenario format pertarungan tunggal 1 vs 1 menghadapi bot asisten yang telah dimodifikasi. Pengujian mencakup pencatatan kejadian-kejadian unik yang muncul selama simulasi pertempuran.

4.3.1 Konfigurasi Lingkungan Pengujian

- **Profil Lawan:** Bot Asisten Kelas (1 vs 1)
- **Turn Limit Per Ronde:** 5000 turn
- **Jumlah Total Rounds:** 10
- **Dimensi Ukuran Arena:** 800x800

4.3.2 Data Hasil Rekapitulasi Match Pengujian

Tabel 4.2: Rekap Hasil Komponen Skor Pengujian 1 s.d 3

Komponen Skor	Pengujian 1		Pengujian 2		Pengujian 3	
	Bot Utama	Asisten	Bot Utama	Asisten	Bot Utama	Asisten
Bullet Damage	416	336	656	302	504	286
Bullet Damage Bonus	58	10	101	0	42	27
Survival Score	150	350	300	200	200	300
Last Survival Bonus	30	70	60	40	40	60
Ram Damage	284	220	229	187	238	178
Ram Damage Bonus	0	35	32	59	63	33
Total Skor	939	1021	1380	788	1087	884
Pemenang	Bot Asisten		Tolak Angin Bot		Tolak Angin Bot	

4.3.3 Catatan Kejadian Unik Selama Pertandingan

Selama pengujian berlangsung, terdapat beberapa peristiwa menarik yang terjadi pada bot. Salah satu peristiwa yang paling sering muncul adalah kedua bot saling mendekat dan melakukan tabrakan berulang (adu banteng). Kondisi ini menyebabkan nilai Ram Damage meningkat cukup tinggi pada beberapa ronde.

Peristiwa tersebut terjadi karena strategi greedy yang digunakan cenderung memilih tindakan paling agresif untuk segera memberikan damage terbesar kepada lawan, terutama ketika musuh berada pada jarak dekat. Akibatnya, bot sering mengabaikan positioning yang aman dan lebih memilih terus maju ke arah lawan.

Selain itu, dalam beberapa kondisi bot juga mengalami movement yang kurang optimal ketika berada terlalu dekat dengan dinding arena sehingga arah gerak menjadi terbatas. Hal ini membuat bot lebih mudah terjebak dalam pertarungan jarak dekat dan saling bertabrakan secara terus-menerus.

Walaupun terlihat kurang efisien dari sisi movement, kondisi adu banteng ini justru menghasilkan Ram Damage yang cukup besar dan membantu meningkatkan total skor pada beberapa pengujian.

4.4 Analisis Hasil Pengujian

Berdasarkan hasil pengujian yang telah dilakukan terhadap bot utama TolakAngintBot melawan bot asisten, strategi greedy yang diimplementasikan menunjukkan performa yang cukup baik dan mampu memperoleh skor tinggi secara konsisten pada beberapa percobaan. Strategi greedy yang digunakan berfokus pada pengambilan keputusan berdasarkan kondisi energi bot serta perilaku agresif untuk memaksimalkan damage terhadap lawan.

Pada Pengujian 1, TolakAngintBot memperoleh Bullet Damage sebesar 416 dan Ram Damage sebesar 284, yang menunjukkan bahwa bot sangat agresif dalam menyerang lawan. Namun, skor survival relatif rendah dibandingkan lawan sehingga total skor masih kalah dari Bot Asisten. Hal ini menunjukkan bahwa strategi greedy agresif mampu menghasilkan damage tinggi, tetapi masih memiliki kelemahan pada aspek pertahanan dan ketahanan hidup.

Pada Pengujian 2, performa bot meningkat secara signifikan dengan total skor mencapai 1380. Peningkatan ini dipengaruhi oleh Bullet Damage yang sangat tinggi yaitu 656 serta Bullet Damage Bonus sebesar 101. Strategi greedy berhasil bekerja optimal karena bot mampu menjaga keseimbangan antara menyerang dan bertahan hidup. Pada kondisi ini, TolakAngintBot mampu memenangkan lebih banyak ronde dibandingkan lawan.

Pada Pengujian 3, TolakAngintBot kembali unggul dengan total skor 1087. Walaupun survival score tidak terlalu tinggi, bot tetap mampu menghasilkan Bullet Damage besar sebesar 504 dan Ram Damage sebesar 238. Hal ini menunjukkan bahwa strategi greedy masih efektif dalam menghasilkan tekanan agresif terhadap lawan.

Secara keseluruhan, strategi greedy yang digunakan dapat dikatakan berhasil karena mampu menghasilkan damage tinggi secara konsisten dan memenangkan beberapa pertandingan. Strategi ini bekerja optimal pada kondisi:

ketika energi bot masih tinggi, ketika lawan berada pada jarak dekat hingga menengah, dan ketika arena tidak terlalu padat oleh banyak bot lain.

Namun, strategi greedy juga memiliki beberapa kelemahan sehingga terkadang menghasilkan kondisi sub-optimal, yaitu:

saat energi bot mulai rendah sehingga movement menjadi kurang agresif, saat menghadapi bot dengan survival movement yang baik, saat lawan lebih sering bertahan di sudut arena, serta ketika bot terlalu fokus menyerang sehingga survival score menurun.

Selain itu, strategi agresif juga membuat bot lebih rentan menerima damage besar dari lawan. Oleh karena itu, keseimbangan antara agresivitas, movement, dan survival menjadi faktor penting agar strategi greedy dapat menghasilkan skor optimal secara konsisten.

BAB 5 Kesimpulan dan Saran

5.1 Kesimpulan

Robocode Tank Royale dapat dimodelkan sebagai persoalan pengambilan keputusan bertahap yang cocok menggunakan pendekatan greedy. Setiap turn, bot memilih aksi terbaik secara lokal berdasarkan informasi yang tersedia. Empat alternatif strategi yang dieksplorasi adalah target terdekat, finisher energi terendah, survival dan jaga jarak, serta skor gabungan. Strategi skor gabungan dipilih sebagai strategi utama karena paling sesuai dengan fungsi objektif permainan, yaitu memaksimalkan skor total melalui keseimbangan antara damage, eliminasi, dan kemampuan bertahan hidup.

5.2 Saran

Pengembangan berikutnya dapat dilakukan dengan menambahkan prediksi posisi target, penyesuaian bobot otomatis berdasarkan kondisi pertandingan, serta pencatatan statistik hasil battle untuk mengevaluasi efektivitas strategi secara kuantitatif. Pengujian juga sebaiknya dilakukan terhadap lebih banyak jenis bot asisten agar kelemahan strategi dapat terlihat lebih jelas.’

Bibliografi

- [1] David Churchill. Portfolio greedy search and simulation for large-scale combat in starcra. https://www.researchgate.net/publication/261527070_Portfolio_greedy_search_and_simulation_for_large-scale_combat_in_starcraft. [Online; Diakses Mei 2026].
- [2] David Churchill and Michael Buro. Portfolio greedy search and simulation for large-scale combat in starcraft. <https://www.researchgate.net/publication/261527070>. [Online; Diakses Mei 2026].
- [3] García Journal of Inequalities and Applications. Greedy algorithms: a review and open problems. <https://www.sciencedirect.com/science/article/abs/pii/S0377221700002526>. [Online; Diakses Mei 2026].
- [4] Journal of Computer System and Informatics (JoSYC). Strategi maximum profit algoritma greedy dalam kecerdasan buatan penentu aktivitas fisik pada mobilexergame. <https://ejurnal.seminar-id.com/index.php/josyc/article/view/3524>. [Online; Diakses Mei 2026].
- [5] Yizhun Wang. Review on greedy algorithm. https://www.researchgate.net/publication/376076453_Review_on_greedy_algorithm. [Online; Diakses Mei 2026].

BAB A Lampiran

A.1 Tautan Repository GitHub

Repository kode sumber tugas besar kelompok kami dapat diakses melalui:

- https://github.com/dinthamdi/Tubes1_Kacauww.git